

人机协作报告：纽约出租车数据分析系统

一、AI 交互日志

1.1 数据处理

Prompt:

加载文件'taxi_zone_lookup.csv'的数据，进行缺失率、异常值统计

生成数据质量报告

去除缺失数据和异常数据

AI 输出：生成了完整的 Python 脚本，包含数据加载、缺失值统计（含热力图和条形图）、IQR/Z-score 异常值检测、数据清洗流程。

我的决策：

A.采用：整体框架、IQR 方法的策略注释、dropna() 删除缺失值的逻辑；

AI 犯错案例 1：

错误：代码中使用了 `fig, ax = plt.subplots(figsize=(10, 5))` 但 `matplotlib` 报错 `'module 'matplotlib' has no attribute 'subplots'`

发现方式：运行时报错 `'AttributeError'`

原因分析：matplotlib 版本兼容性问题，部分旧版本不支持此简写

修正方法：改为 `plt.figure(figsize=(10, 5))` 并添加 `matplotlib.use('Agg')` 设置非交互式后端

1.2 可视化分析

交互过程：逐步确认数据集列名、字段含义，根据实际数据调整分析维度。

我的决策：

A.修改：基础车费数值远大于其他费用导致 z 标签位置过高，将数据标签从柱子上方移到图表顶部；

B.拒绝：去除了缺失值热力图。

AI 犯错案例 2：

错误：数据清洗后创建 `df_cleaned`，但可视化代码中部分引用了 `df_cleaned`，部分引用了 `data`，导致 `'NameError'`

发现方式：运行时报错

原因分析：分拆文件时变量命名不一致，AI 未检查跨文件变量引用

修正方法：统一将可视化文件中的 `df_cleaned` 全部替换为 `data`

AI 犯错案例 3：

错误：分组操作 `data.groupby('pickup_date')` 报 `'KeyError: 'pickup_date''`

发现方式：运行时异常

原因分析：时间特征提取未成功执行，`'pickup_date'` 列实际上未被创建

修正方法：改用 `data.groupby(data['pickup\_datetime'].dt.date)` 直接提取日期分组，避开中间列问题

1.3 预测模型

Prompt:

用 TensorFlow 构建神经网络，预测某区域某时段的出行需求量；

按 8:2 划分训练集和测试集，绘制 loss 曲线和 MAE 曲线，测试集报告 MAE 和 RMSE；

将预测模型与随机森林进行对比，生成 MAE 对比，RMSE 对比和残差对比

AI 输出：完整的神经网络和随机森林对比，包含训练、评估、可视化。

我的决策：

修改：消除 TensorFlow oneDNN 和 absl 日志警告。

1.4 问答系统

交互过程：指定 6 种问题类型，由 AI 生成自然语言理解（正则匹配、意图识别）和规则回答。

二、三阶段对比：以「区域时段需求量查询」为例

2.1 Native 版（独立写）

```
```python
def query_hourly_demand():
 hourly = data.groupby('pickup_hour').size()
 print(hourly)
...
```
```

特点：输出为文本数字，无图表，无参数化。需手动修改代码切换区域或日期类型。

效率评估：开发时间约 5 分钟（含调试），但每次换查询条件都需重写代码。理解深度最高，因为必须清楚每一行逻辑。

2.2 Prompt 版（AI 辅助）

Prompt：“写一个函数，输入区域 ID（可选）和日期类型（工作日/周末/全部），输出分小时订单量柱状图，标注峰值”

```
```python
def query_hourly_demand(location_id=None, day_type='全部'):
 query_data = data if location_id is None else data[data['PULocationID'] == location_id]
 if day_type == '工作日':
 query_data = query_data[~query_data['is_weekend']]
 elif day_type == '周末':
 query_data = query_data[query_data['is_weekend']]
...
```
```

```

hourly = query_data.groupby('pickup_hour').size()
hourly = hourly.reindex(range(24), fill_value=0)

fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(hourly.index, hourly.values, color='steelblue')
# ... 完整图表代码
plt.savefig('output.png')
return {'peak_hour': hourly.idxmax(), 'chart': 'output.png'}
...

```

特点：含参数化、图表保存、结构化返回值。AI 自动补充了 `reindex` 填充缺失时段、中文字体设置等细节。

效率评估：不到分钟获得可用代码，再花 3 分钟调整参数和路径，代码质量显著高于 Native 版。

2.3 Vibe 版（对话驱动）

代码：

```

def query_hourly_demand(location_id=None, day_type='全部'):
    """M4-1: 时段需求量查询"""
    if location_id:
        query_data = data[data['PULocationID'] == int(location_id)]
        title_loc = f"区域 {location_id}"
    else:
        query_data = data
        title_loc = "全区域"

    if day_type == '工作日':
        query_data = query_data[~query_data['is_weekend']]
        title_type = '工作日'
    elif day_type == '周末':
        query_data = query_data[query_data['is_weekend']]
        title_type = '周末'
    else:
        title_type = '全部'

    hourly = query_data.groupby('pickup_hour').size()
    hourly = hourly.reindex(range(24), fill_value=0)

    # 绘图
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.bar(hourly.index, hourly.values, color='steelblue', edgecolor='white', alpha=0.8)
    ax.plot(hourly.index, hourly.values, 'ro-', markersize=5, linewidth=2)
    ax.set_xlabel('小时', fontsize=12)
    ax.set_ylabel('订单量', fontsize=12)

```

```

ax.set_title(f'{title_loc} - {title_type} 分时段需求量', fontsize=14, fontweight='bold')
ax.set_xticks(range(0, 24, 2))
ax.grid(axis='y', alpha=0.3)

peak_hour = hourly.idxmax()
peak_value = hourly.max()

filepath = 'outputs/qa_hourly_demand.png'
plt.tight_layout()
plt.savefig(filepath, dpi=150, bbox_inches='tight')
plt.close()

return {
    'peak_hour': peak_hour,
    'peak_value': int(peak_value),
    'avg_demand': round(hourly.mean(), 1),
    'morning_peak': f'{hourly[6:10].idxmax()}时 ({hourly[6:10].max()}单)',
    'evening_peak': f'{hourly[16:20].idxmax()}时 ({hourly[16:20].max()}单)',
    'chart': filepath
}

```

效率评估：总耗时约 8 分钟（多轮对话），但每轮修改都很小（1-2 行）。优势在于渐进式需求细化，不需要一次性想清楚所有功能；劣势在于轮次较多时上下文可能丢失。

三者对比：

代码质量依次递增，但理解深度依次递减

三、反思：对 AI 工具能力边界的新认识

经过本次完整的数据分析系统开发，我对 AI 辅助编程有了更深入的认识。

AI 擅长的是“翻译”而非“创造”。当我能清晰描述需求时，AI 能将需求准确转化为代码；但当我自己也不确定该怎么做时，AI 给出的答案往往是缺乏针对性。这说明 AI 工具是强执行者、弱决策者。

AI 对上下文的理解能力一般。在过程中，AI 出现了分拆文件时变量名不统一、索引超限不主动提示等问题，说明 AI 不擅长跨模块的一致性检查。它在单函数、单文件的局部范围内表现出色，但全局架构的协调仍是人类的职责。

错误诊断的效率较低。AI 的一些低级语法错误可以秒立即修改，但运行时异常往往需要多轮排查。例如 `matplotlib.subplots` 报错，AI 连续两次修复方案都未命中根本原因。

对比三阶段编码模式，效率差异不在于 AI 本身，而在于我是否清楚地知道并表达需求。当需求明确时，一个 Prompt 就能获得高质量代码；当需求模糊时，Vibe Coding 虽能逐步收敛，但前提是我能识别每一次迭代中的偏差并纠正。

总而言之，我认为当前 AI 工具的最佳角色是“助手”而非“替代开发者”。它极大降低了

重复性编码和查文档的成本，但架构设计、需求分析、质量把关这些核心任务仍需要人的判断力和领域知识。使用 AI 的正确姿势不是放手让它全权负责，而是像合作伙伴一样频繁检查、及时纠偏、持续微调。